

Universidad de las Américas

Facultad de Ingeniería y Ciencias Aplicadas

Programación III

NRC 2336

Docente: Byron Rodríguez

Avance del Proyecto – Progreso 1

Análisis, diseño y selección de la solución

Simulador de gestión de pistas de un aeropuerto

mediante colas de prioridad y pilas

Integrantes:

Nicole Chancusig

David Galarza

Felipe Hurtado

Mateo Izquierdo

Quito, 27 de octubre de 2026

Índice

1. Formulación del problema....3
1.1 Contexto y necesidad....3
1.2 Actores involucrados....3
1.3 Causas y consecuencias....3
1.4 Datos y procesos principales....4
1.5 Variables relevantes....4
1.6 Restricciones....5
1.7 Relación con factores globales....5
1.8 Alcance....5
2. Alcance y módulos funcionales....6
3. Modelo orientado a objetos y diagrama UML preliminar....9
3.1 Clases de dominio....9
3.2 Enumeraciones....10
3.3 Relaciones y multiplicidades....10
3.4 Diagrama de clases....11
4. Alternativas basadas en estructuras de datos y algoritmos....12
4.1 Alternativa A: cola de prioridad con heap binario....12
4.2 Alternativa B: colas FIFO por nivel con envejecimiento....12
4.3 Cuadro comparativo....13
5. Selección y fundamentación de la mejor alternativa....14
6. Plan preliminar de implementación....15
6.1 Módulos asignados por integrante....15
6.2 Estructura inicial del repositorio....15
6.3 Cronograma hacia el Progreso 2....16
6.4 Riesgos técnicos y mitigación....16
Declaración de uso de herramientas de inteligencia artificial....17
Referencias....17
Anexo A. Código PlantUML del diagrama de clases....19

1. Formulación del problema

1.1 Contexto y necesidad

En un aeropuerto, la pista es el recurso más escaso y el más difícil de ampliar. Cada aterrizaje y cada despegue debe ocurrir en un orden y en un momento concretos, respetando tiempos mínimos de separación entre aeronaves. En la literatura, este problema se conoce como programación de pistas y, en su variante de llegadas, como problema de aterrizaje de aeronaves: consiste en secuenciar las aeronaves en las pistas y programar sus tiempos de operación considerando varias restricciones operativas (Bennell et al., 2011; Shirini et al., 2024). Además, cada avión debe operar dentro de una ventana de tiempo predeterminada y respetar criterios de separación con los aviones que lo siguen (Beasley et al., 2000).

En Quito, el Aeropuerto Internacional Mariscal Sucre funciona con una sola pista de 4 100 m y en 2023 recibió cerca de 5,5 millones de pasajeros (Odinsa, s.f.). Con una sola pista, el orden en que se atiende a cada avión afecta a todas las operaciones que vienen después.

En la práctica, el controlador debe decidir quién usa la pista primero cuando varios aviones la piden a la vez, y no todos tienen la misma urgencia: una emergencia o un vuelo con poco combustible no puede esperar como un vuelo regular. Cuando esta decisión no sigue un criterio sistemático, aumentan los retrasos, se consume combustible en esperas y se compromete la seguridad. Por eso hace falta una herramienta que ordene las solicitudes con reglas claras, asigne la pista y permita medir cuánto esperan los vuelos con esas reglas.

1.2 Actores involucrados

- **Controlador de tránsito aéreo (actor principal):** registra solicitudes, declara emergencias y ejecuta la asignación de pistas.
- **Administrador del aeropuerto:** habilita o cierra pistas, define los parámetros y consulta estadísticas.
- **Aerolíneas y tripulaciones:** originan las solicitudes, reportan el combustible y declaran emergencias.
- **Pasajeros:** no usan el sistema, pero sufren los retrasos y dependen de la seguridad de las operaciones.

1.3 Causas y consecuencias

El problema se produce así: la demanda de operaciones crece mientras el número de pistas es fijo, por lo que varias aeronaves solicitan pista al mismo tiempo y se forma una cola de espera. Como entre una operación y la siguiente debe respetarse un tiempo mínimo de separación, la pista no puede atender más rápido aunque haya más solicitudes. El resultado es un mayor tiempo de espera por aeronave, menor aprovechamiento de la pista y más costo de

combustible por maniobras y tiempo de vuelo adicional (Mesgarpour, 2012). En los casos críticos, un vuelo con combustible bajo o con una emergencia no puede esperar su turno, lo que eleva el riesgo operativo; por eso la literatura plantea como objetivos de una buena programación reducir los retrasos y la contaminación (Shirini et al., 2024).

Además, si no hay un criterio fijo para desempatar, dos controladores pueden ordenar de forma distinta las mismas solicitudes, y una solicitud regular puede quedarse esperando mucho tiempo si siempre llegan otras más urgentes.

1.4 Datos y procesos principales

Cada solicitud contiene el identificador de vuelo, la aerolínea, el tipo de operación (aterrizaje o despegue), el nivel de prioridad (emergencia, combustible bajo o regular), la hora de solicitud y el combustible restante, en minutos de autonomía. Cada pista tiene un identificador, un estado, las operaciones que admite y la última operación realizada. El proceso general es: (1) el avión solicita pista; (2) el sistema registra y clasifica la solicitud; (3) la inserta en la cola según su prioridad; (4) cuando hay una pista compatible disponible, asigna la solicitud de mayor prioridad; (5) se ejecuta la operación y se libera la pista; y (6) se registran el tiempo de espera y la operación en el historial.

1.5 Variables relevantes

Tabla 1
Variables del modelo

Variable	Descripción	Tipo / unidad
Nivel de prioridad	Emergencia, combustible bajo o regular	Enumeración
Tipo de operación	Aterrizaje o despegue	Enumeración
Hora de solicitud	Minuto simulado en que llega la solicitud	Entero (min)
Tiempo de espera	Hora de asignación menos hora de solicitud	Entero (min)
Combustible restante	Autonomía de vuelo reportada por la aeronave	Entero (min)
Número de pistas (p)	Pistas habilitadas en la simulación	Entero
Estado de pista	Libre, ocupada o cerrada	Enumeración
Tiempo de ocupación	Minutos que una operación ocupa la pista	Entero (min)
Separación mínima	Tiempo entre inicios de operaciones consecutivas en una pista	Entero (min)

1.6 Restricciones

Los valores son simulados: los definió el equipo y se guardan en un archivo de configuración para poder cambiarlos. **Restricciones de la situación:**

- Una pista atiende una sola operación a la vez.
- Separación mínima entre inicios de operaciones consecutivas en una pista: 3 minutos para aterrizaje tras aterrizaje y 2 minutos en cualquier otra combinación.
- Ocupación de pista: 2 minutos por aterrizaje y 1 minuto por despegue.
- Las emergencias tienen precedencia absoluta sobre cualquier otra solicitud.
- Una autonomía menor a 20 minutos se clasifica como combustible bajo; aplica solo a aterrizajes, porque una aeronave en tierra no corre ese riesgo.
- Una solicitud regular que espera 15 minutos o más sube a combustible bajo para no esperar indefinidamente; nunca supera a una emergencia.
- Las solicitudes llegan de forma dinámica y el tiempo avanza en pasos de un minuto simulado.

Restricciones del proyecto:

- Java 17, interfaz de consola y datos en memoria.
- El heap y la pila se implementan desde cero; `java.util.PriorityQueue` solo se usa en las pruebas para comparar.
- Plazo del Progreso 2 y cuatro integrantes.

1.7 Relación con factores globales

Existe una relación económica y ambiental justificable: las esperas generan costos operativos para las aerolíneas y emisiones adicionales por vuelos en espera (Mesgarpour, 2012; Shirini et al., 2024). También tiene un componente social, porque las reglas de prioridad protegen a las personas que viajan en un vuelo con emergencia. No se identifica una relación cultural directa, por lo que no se incluye.

1.8 Alcance

El proyecto consiste en un simulador en Java que gestiona las solicitudes con una cola de prioridad, asigna pistas y mide los tiempos de espera. No se busca la programación óptima, que en la literatura se resuelve con programación matemática y metaheurísticas (Bennell et al., 2011).

2. Alcance y módulos funcionales

El sistema simula la gestión de solicitudes de pista de un aeropuerto: los aviones piden aterrizar o despegar con un nivel de prioridad, el sistema los ordena, les asigna una pista disponible y mide los tiempos de espera. Se definen cinco módulos funcionales. Además de las operaciones CRUD que hagan falta, cada módulo tiene operaciones propias del problema, como búsqueda, filtrado, priorización, asignación u ordenamiento.

Módulo 1: Gestión de aeronaves y solicitudes

- **Propósito y actor:** registrar los aviones y sus solicitudes de operación. Actor principal: controlador.
- **Datos:** identificador de vuelo, aerolínea, tipo de aeronave, tipo de operación, nivel de prioridad, combustible restante y hora de solicitud.
- **Operaciones:** (1) registrar una solicitud; (2) modificar o cancelar una solicitud; (3) buscar un vuelo por identificador en O(1) mediante un HashMap; (4) filtrar solicitudes por aerolínea, tipo de operación o prioridad; (5) clasificar automáticamente una solicitud según su prioridad.
- **Reglas del negocio:** un vuelo no puede tener dos solicitudes activas a la vez. Un aterrizaje con autonomía por debajo del umbral se clasifica como combustible bajo. Una emergencia declarada se clasifica como emergencia.
- **Relación con otros módulos:** entrega las solicitudes al Módulo 3 y consulta al Módulo 4 el estado de combustible.
- **Clases:** `Aeronave`, `Solicitud`, `SolicitudAterrizaje`, `SolicitudDespegue`, `RegistroSolicitudes`.
- **Responsable preliminar:** Nicole Chancusig.

Módulo 2: Gestión de pistas

- **Propósito y actor:** administrar el estado de cada pista y su disponibilidad. Actores: controlador y administrador.
- **Datos:** identificador de pista, estado (libre, ocupada o cerrada), operaciones permitidas, última operación, hora de inicio de la última operación y hora de liberación.
- **Operaciones:** (1) registrar o habilitar una pista; (2) cerrar o reabrir una pista; (3) buscar la primera pista disponible y compatible con la operación (búsqueda lineal, O(p)); (4) calcular la hora en que la pista queda disponible según la separación requerida; (5) liberar una pista al terminar la operación.

- **Reglas del negocio:** una pista atiende una sola operación a la vez. Entre dos operaciones consecutivas debe haber el tiempo mínimo de separación, que depende de la operación anterior y de la nueva. Una pista cerrada no puede recibir asignaciones.
- **Relación con otros módulos:** el Módulo 3 le consulta la disponibilidad y le asigna operaciones; informa al Módulo 5 el uso de cada pista.
- **Clases:** `Pista`.
- **Responsable preliminar:** David Galarza.

Módulo 3: Cola de prioridad y asignación de pistas (módulo central)

- **Propósito y actor:** ordenar las solicitudes pendientes y asignarlas a pistas. Actor principal: controlador.
- **Datos:** cola de solicitudes pendientes, prioridad, hora de solicitud, pista asignada y horas de inicio y fin de cada operación.
- **Operaciones:** (1) encolar una solicitud según su prioridad; (2) extraer la solicitud de mayor prioridad; (3) asignar la solicitud a una pista disponible; (4) reordenar la cola cuando cambia la prioridad de un vuelo; (5) aplicar el envejecimiento a las solicitudes regulares; (6) consultar el tamaño y el contenido de la cola.
- **Reglas del negocio:** la precedencia es emergencia, luego combustible bajo y luego vuelo regular. Dentro de combustible bajo va primero la aeronave cuyo combustible se agota antes. A igual prioridad tiene preferencia la solicitud con la hora de solicitud más antigua, es decir, la de mayor tiempo de espera. Una solicitud regular no puede esperar indefinidamente: al llegar al límite de espera sube de nivel.
- **Relación con otros módulos:** recibe solicitudes del Módulo 1, usa el Módulo 2 para asignar y envía datos a los Módulos 4 y 5.
- **Clases:** `ColaPrioridad`, `Asignacion`, `Aeropuerto`.
- **Responsable preliminar:** Felipe Hurtado, con apoyo de todo el equipo en la integración.

Módulo 4: Emergencias y control de combustible

- **Propósito y actor:** vigilar el combustible y gestionar los casos urgentes. Actor principal: controlador.
- **Datos:** nivel de combustible, umbral de alerta, tipo de emergencia, hora de declaración y alertas emitidas.
- **Operaciones:** (1) declarar una emergencia; (2) actualizar el combustible de un vuelo; (3) detectar los vuelos bajo el umbral y escalar su prioridad; (4) listar los vuelos críticos ordenados por urgencia; (5) generar y registrar una alerta.

- **Reglas del negocio:** una emergencia pasa por delante de toda la cola. Si el combustible baja del umbral, la prioridad sube automáticamente y la solicitud se reubica en el heap. Toda alerta emitida se registra.
- **Relación con otros módulos:** modifica prioridades en el Módulo 3 y lee datos del Módulo 1.
- **Clases:** `ControlCombustible`, `Alerta`.
- **Responsable preliminar:** Mateo Izquierdo.

Módulo 5: Estadísticas, historial y reportes

- **Propósito y actor:** medir el desempeño y guardar el historial de operaciones. Actor principal: administrador.
- **Datos:** tiempos de espera, operaciones completadas, historial reciente y conteos por prioridad y por pista.
- **Operaciones:** (1) calcular el tiempo de espera promedio y máximo; (2) registrar cada operación completada en el historial (pila); (3) consultar las últimas k operaciones; (4) generar un reporte por tipo de prioridad o por pista; (5) ordenar los vuelos por tiempo de espera con merge sort, O(n log n).
- **Reglas del negocio:** toda operación completada se registra una sola vez. Las estadísticas se calculan solo con operaciones terminadas.
- **Relación con otros módulos:** recibe datos de los Módulos 2, 3 y 4.
- **Clases:** `HistorialOperaciones`, `PilaHistorial`, `Estadisticas`.
- **Responsable preliminar:** Nicole Chancusig y David Galarza, de forma compartida.

3. Modelo orientado a objetos y diagrama UML preliminar

Las clases salen de los módulos funcionales: cada módulo tiene al menos una clase propia y la clase `Aeropuerto` coordina a las demás. El diagrama se hizo en PlantUML y lo revisó el equipo. Su código está en el Anexo A y en la carpeta `docs/uml` del repositorio.

3.1 Clases de dominio

Tabla 2
Clases de dominio, responsabilidades y módulo al que pertenecen

Clase	Responsabilidad y atributos principales	Métodos relevantes	Mód.
Aeronave	Datos del vuelo: idVuelo, aerolinea, tipo, combustibleRestante, horaReporteCombustible	actualizarCombustible(), horaAgotamiento(), estaBajoUmbral()	1 y 4
Solicitud (abstracta)	Pedido de pista: aeronave, prioridad, prioridadOriginal, horaSolicitud, secuencia	getTipo(), separacionRequerida(anterior), calcularEspera(), subirPrioridad(), compareTo()	1 y 3
SolicitudAterrizaje	Hereda de Solicitud; es la única que puede clasificarse como combustible bajo	separacionRequerida(anterior): 3 min tras un aterrizaje, 2 min en otro caso	1
SolicitudDespegue	Hereda de Solicitud	separacionRequerida(anterior): 2 min	1
RegistroSolicitudes	Índices HashMap por idVuelo de aeronaves y de solicitudes activas	registrar(), cancelar(), buscarPorVuelo(), filtrar(), clasificar()	1
Pista	id, estado, operacionesPermitidas, ultimaOperacion, horaUltimoInicio, horaLiberacion	estaDisponible(), horaDisponible(), ocupar(), liberar(), cerrar(), reabrir()	2
ColaPrioridad	Heap binario sobre un arreglo de solicitudes	encolar(), extraer(), verSiguiente(), reordenar(), eliminar(), tamano()	3
Asignacion	solicitud, pista, horaInicio, horaFin	duracion(), tiempoEsperado()	3
Aeropuerto (controlador)	horaActual; coordina pistas, cola, registro, control de combustible, historial y estadísticas	recibirSolicitud(), asignarSiguiente(), avanzarTiempo(), aplicarEnvejecimiento()	Todos
ControlCombustible	umbralMinutos y registro de alertas	declararEmergencia(), actualizarCombustible(), detectarBajoUmbral(), listarCriticos(), generarAlerta()	4
Alerta	tipo, aeronave, hora, descripcion	Objeto de datos	4

Clase	Responsabilidad y atributos principales	Métodos relevantes	Mód.
HistorialOperaciones	Historial de asignaciones completadas sobre una pila	registrar(), consultarUltimas(k), reportePorPista()	5
PilaHistorial<T>	Pila genérica con nodos enlazados: tope, tamano	apilar(), desapilar(), verTope(), ultimos(k)	5
Estadisticas	totalOperaciones, sumaEspera, maxEspera, conteoPorPrioridad	registrar(), promedioEspera(), reportePorPrioridad(), ordenarPorEspera()	5

3.2 Enumeraciones

- `Prioridad`: EMERGENCIA, COMBUSTIBLE_BAJO, REGULAR. El orden en que se declaran los valores define la precedencia.
- `TipoOperacion`: ATERRIZAJE, DESPEGUE.
- `EstadoPista`: LIBRE, OCUPADA, CERRADA.
- `TipoAlerta`: EMERGENCIA, COMBUSTIBLE_BAJO, ESPERA_EXCEDIDA.

3.3 Relaciones y multiplicidades

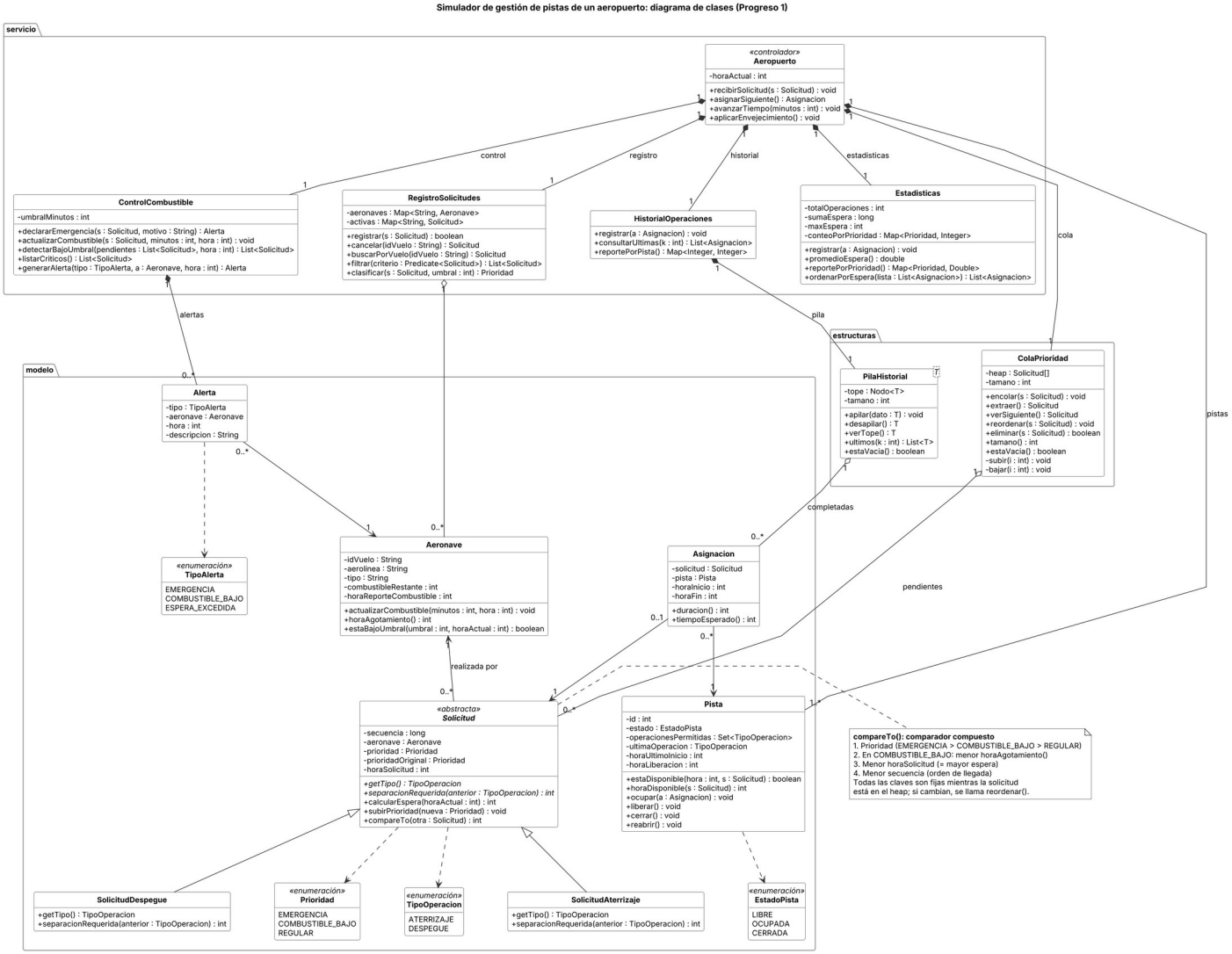
- **Herencia:** `Solicitud` (abstracta) es generalizada por `SolicitudAterrizaje` y `SolicitudDespegue`. La herencia tiene sentido porque cada tipo calcula de forma distinta la separación que necesita respecto de la operación anterior (método polimórfico `separacionRequerida`) y porque solo los aterrizajes pueden clasificarse como combustible bajo.
- **Composición:** el Aeropuerto contiene una o más Pistas (1..*) y una instancia de ColaPrioridad, RegistroSolicitudes, ControlCombustible, HistorialOperaciones y Estadisticas. Estas partes no existen fuera del aeropuerto. Del mismo modo, HistorialOperaciones contiene su PilaHistorial y ControlCombustible contiene sus Alertas (0..*).
- **Agregación:** la ColaPrioridad agrupa cero o más Solicitudes (0..*), que existen antes de entrar a la cola y siguen existiendo al salir. Lo mismo ocurre entre RegistroSolicitudes y Aeronave, y entre PilaHistorial y Asignacion.
- **Asociación:** una Aeronave (1) tiene cero o más Solicitudes (0..*) a lo largo del tiempo, pero solo una activa a la vez; una Solicitud (1) tiene cero o una Asignacion (0..1); una Pista (1) tiene cero o más Asignaciones (0..*); y una Aeronave (1) puede tener cero o más Alertas (0..*).

El diagrama completo se presenta en la página siguiente (Figura 1).

3.4 Diagrama de clases

Figura 1

Diagrama de clases preliminar del simulador



Nota. Elaboración propia en PlantUML. Rombo relleno: composición; rombo vacío: agregación; triángulo vacío: herencia; flecha punteada: dependencia. Código en el Anexo A.

4. Alternativas basadas en estructuras de datos y algoritmos

Se comparan dos alternativas para el componente central del sistema: ordenar las solicitudes pendientes, elegir la siguiente a atender y asignarla a una pista. Sea *n* el número de solicitudes pendientes, *m* el número de operaciones registradas en el historial y *p* el número de pistas, que es pequeño y fijo.

4.1 Alternativa A: cola de prioridad con heap binario

Se mantiene un heap binario sobre un arreglo, ordenado por un comparador compuesto de cuatro claves: (1) nivel de prioridad; (2) dentro de combustible bajo, la hora estimada en que se agota el combustible (hora del reporte más autonomía); (3) la hora de solicitud más antigua, que equivale a la de mayor tiempo de espera; y (4) un número de secuencia de llegada que hace el orden total y estable. Las inserciones y extracciones usan *sift-up* y *sift-down* en O(log n) (Cormen et al., 2022). La asignación es voraz: se toma la solicitud de la cima y se recorre la lista de pistas para encontrar la primera disponible y compatible; si no hay ninguna, la solicitud permanece en la cola hasta el siguiente minuto simulado.

Las cuatro claves tienen que ser fijas mientras la solicitud está en el heap. Si se ordenara por el tiempo de espera calculado en cada momento, ese valor cambiaría con el reloj y el heap dejaría de estar ordenado. Por eso se usa la hora de solicitud, que da el mismo orden y no cambia. Con el combustible pasa algo parecido: todas las aeronaves en vuelo gastan un minuto de autonomía por cada minuto, así que la hora de agotamiento se mantiene igual hasta que llega un reporte nuevo; en ese caso se llama a `reordenar()`.

Tabla 3

Ejemplo de orden de atención con el comparador compuesto (hora actual 10:06)

Vuelo	Operación	Prioridad	Hora de solicitud	Agota combustible	Orden
EQ303	Aterrizaje	Emergencia	10:05	No aplica	1
KL404	Aterrizaje	Combustible bajo	10:03	10:18	2
AV202	Aterrizaje	Combustible bajo	10:01	10:22	3
AA505	Despegue	Regular	09:58	No aplica	4
LA101	Aterrizaje	Regular	10:00	No aplica	5

Nota. KL404 se atiende antes que AV202 aunque llegó después, porque su combustible se agota antes. Con la Alternativa B ambos estarían en la misma cola FIFO y AV202 saldría primero. Entre los regulares, AA505 va antes que LA101 porque lleva más tiempo esperando.

4.2 Alternativa B: colas FIFO por nivel con envejecimiento

Se mantienen tres colas FIFO implementadas con listas enlazadas: emergencia, combustible bajo y regular. Se atiende siempre la primera cola no vacía, en orden de llegada. El

envejecimiento solo necesita revisar el frente de la cola regular: como la cola es FIFO, las solicitudes más antiguas están adelante, así que se promueven a la cola superior mientras la del frente supere el límite de espera. Esta técnica de envejecimiento es la misma que usan los sistemas operativos en la planificación por prioridad para evitar la inanición (Silberschatz et al., 2018).

4.3 Cuadro comparativo

Tabla 4
Comparación técnica de las alternativas

Aspecto	Alternativa A: heap binario	Alternativa B: colas FIFO por nivel
Estructura(s)	Heap binario sobre arreglo (cola de prioridad); pila enlazada para el historial; HashMap de solicitudes por idVuelo	Tres colas FIFO con listas enlazadas; pila enlazada para el historial; HashMap de solicitudes por idVuelo
Algoritmo(s)	Inserción y extracción con sift-up y sift-down; comparador compuesto de 4 claves; asignación voraz con búsqueda lineal de pista; envejecimiento por recorrido	Atención por niveles; envejecimiento revisando el frente de la cola regular; asignación voraz con búsqueda lineal de pista
Operación que resuelve	Ordenar las solicitudes por varios criterios y elegir la siguiente a atender	Ordenar las solicitudes por nivel y evitar la inanición
Complejidad temporal	Encolar O(log n) Extraer O(log n) Ver la siguiente O(1) Cambiar prioridad o cancelar: O(n) localizar + O(log n) reubicar; O(log n) con índice auxiliar Envejecimiento O(n) por ciclo Buscar pista O(p)	Encolar O(1) Extraer O(1) (máximo 3 colas) Ver la siguiente O(1) Cambiar prioridad o cancelar: O(n) localizar + O(1) mover Envejecimiento O(1 + k) por ciclo, k = promovidas Buscar pista O(p)
Complejidad espacial	O(n) para el heap + O(m) para el historial	O(n) para las colas, con un puntero extra por nodo + O(m) para el historial
Ventajas	Ordena con criterios finos (prioridad, combustible y espera) en una sola estructura; agregar un criterio solo requiere cambiar compareTo()	Operaciones básicas en tiempo constante; muy fácil de implementar y depurar; envejecimiento barato
Limitaciones	Cambiar la prioridad de un elemento exige localizarlo; el heap no es estable por sí solo (se resuelve con la clave de secuencia)	Solo distingue tres niveles fijos: dentro de un nivel no puede ordenar por combustible sin perder el O(1), porque insertar ordenado costaría O(n); agregar criterios exige más colas y reglas de paso

Nota. Las complejidades del heap binario siguen a Cormen et al. (2022). La documentación de `java.util.PriorityQueue` confirma O(log n) para encolar y extraer, y tiempo lineal para eliminar un objeto arbitrario (Oracle, s.f.). Referencia general de estructuras de datos en Java: Aguas Bucheli (2024).

5. Selección y fundamentación de la mejor alternativa

Para comparar de forma sistemática se usa una matriz de decisión con criterios ponderados. Las ponderaciones y los puntajes, de 1 a 5 donde 5 es mejor, son una valoración del equipo y se justifican después de la tabla.

Tabla 5
Matriz de decisión ponderada

Criterio	Peso	Alternativa A	Alternativa B
Adecuación al problema	25 %	5	3
Tiempo de ejecución esperado	20 %	4	5
Uso de memoria	10 %	4	4
Escalabilidad	15 %	5	3
Mantenibilidad	15 %	4	4
Dificultad razonable de implementación	15 %	3	5
Puntaje ponderado	**100 %**	**4,25**	**3,95**

Nota. A = 0,25 × 5 + 0,20 × 4 + 0,10 × 4 + 0,15 × 5 + 0,15 × 4 + 0,15 × 3 = 4,25. B = 0,25 × 3 + 0,20 × 5 + 0,10 × 4 + 0,15 × 3 + 0,15 × 4 + 0,15 × 5 = 3,95.

Alternativa seleccionada: A (cola de prioridad con heap binario).

En adecuación al problema, A es mejor porque hay que ordenar por varios criterios a la vez: nivel de urgencia, combustible y tiempo de espera. El heap lo resuelve con un solo comparador, mientras que B solo distingue tres niveles fijos y no puede adelantar, dentro de combustible bajo, al vuelo que se queda sin combustible primero (Tabla 3).

En tiempo de ejecución, encolar y extraer en O(log n) es más lento que O(1), pero con el volumen de un aeropuerto simulado la diferencia no se nota: con 500 solicitudes, $\log_2 n$ es casi 9, o sea unas 9 comparaciones por operación. El punto débil de A es cambiar la prioridad o cancelar una solicitud, porque primero hay que encontrarla en O(n). Eso puede bajar a O(log n) con un HashMap que guarde la posición de cada vuelo en el arreglo.

En memoria, las dos usan O(n). El heap sobre arreglo no necesita punteros y las listas enlazadas de B sí, pero la diferencia es solo un factor constante, por eso tienen el mismo puntaje.

En escalabilidad, O(log n) crece muy despacio cuando aumentan las solicitudes, y si se agrega un criterio nuevo basta con cambiar `compareTo()`; en B habría que crear más colas y reglas para pasar de una a otra. En mantenibilidad las dos son claras, pero A tiene toda la lógica de prioridad en un solo método.

En dificultad, B es más fácil de programar. A exige dominar sift-up y sift-down, pero es parte de lo visto en la materia, y el riesgo se controla con pruebas unitarias que comparan nuestro heap con `java.util.PriorityQueue`.

Por último, de B se toma la idea del envejecimiento: en cada ciclo, las solicitudes regulares que llegan al límite de espera suben a combustible bajo y se reubican en el heap con `reordenar()`.

6. Plan preliminar de implementación

6.1 Módulos asignados por integrante

Tabla 6
Asignación preliminar de módulos y componentes

Integrante	Módulo(s)	Componentes de código
Nicole Chancusig	Módulo 1 y parte del 5	Aeronave, Solicitud y subclases, RegistroSolicitudes (búsqueda y filtros), reportes
David Galarza	Módulo 2 y parte del 5	Pista (búsqueda de pista disponible y separación), PilaHistorial, HistorialOperaciones, Estadisticas
Felipe Hurtado	Módulo 3	ColaPrioridad (heap), Asignacion, Aeropuerto (ciclo de simulación y envejecimiento)
Mateo Izquierdo	Módulo 4	ControlCombustible, Alerta, escalamiento de prioridad

La interfaz de consola (`MenuConsola`), la clase `Main` y las pruebas de integración se reparten entre todos en la fase de integración.

6.2 Estructura inicial del repositorio

El repositorio está en https://github.com/bundo123/simulador-aeropuerto. Usa la estructura estándar de Maven y ya tiene el esqueleto de todas las clases del diagrama, con sus atributos y las firmas de sus métodos. Cada integrante trabaja en su propia rama y los cambios se unen a `main` con *pull requests* que revisa otro integrante.

```
simulador-aeropuerto/
├── README.md
├── pom.xml
├── .gitignore
├── config/
│   └── parametros.properties
├── docs/
│   ├── informe/Avance_Progreso1.pdf
│   ├── uml/diagrama-clases.puml y diagrama-clases.png
│   └── presentacion/Presentacion_Progreso1.pdf
└── src/
    ├── main/java/aeropuerto/
    │   ├── Main.java
    │   ├── modelo/
    │   │   ├── Aeronave.java, Solicitud.java,
    │   │   │   SolicitudAterrizaje.java, SolicitudDespegue.java,
    │   │   │   Pista.java, Asignacion.java, Alerta.java
    │   │   └── enums/ Prioridad, TipoOperacion, EstadoPista, TipoAlerta
    │   ├── estructuras/ ColaPrioridad.java, PilaHistorial.java
    │   ├── servicio/ Aeropuerto.java, RegistroSolicitudes.java,
    │   │             ControlCombustible.java, HistorialOperaciones.java,
    │   │             Estadisticas.java
    │   └── ui/ MenuConsola.java
    └── test/java/aeropuerto/
        ├── ColaPrioridadTest.java
        ├── PistaTest.java
        └── AeropuertoTest.java
```

6.3 Cronograma hacia el Progreso 2

Tabla 7
Cronograma preliminar

Semana	Actividades
1	Estructuras propias (heap y pila) con pruebas unitarias; clases del modelo
2	Pistas y separación; registro, búsqueda y filtrado de solicitudes; control de combustible
3	Integración en Aeropuerto: ciclo de simulación, asignación y envejecimiento
4	Estadísticas, historial, menú de consola, pruebas de carga y documentación

6.4 Riesgos técnicos y mitigación

Tabla 8
Riesgos técnicos identificados y acciones de mitigación

Riesgo	Mitigación
Errores al implementar el heap (sift-up y sift-down)	Probar la estructura de forma aislada con casos pequeños y comparar sus resultados con java.util.PriorityQueue antes de integrarla

Riesgo	Mitigación
Claves que cambian mientras la solicitud está en el heap rompen el orden	Usar solo claves fijas (hora de solicitud, hora de agotamiento, secuencia) y llamar a reordenar() cuando una clave cambie
Cambiar la prioridad de un elemento ya encolado cuesta O(n)	Aceptarlo en la primera versión; si es necesario, agregar un HashMap auxiliar de vuelo a posición
Inanición de solicitudes regulares	Implementar la regla de envejecimiento y probarla con casos de carga alta de emergencias
Bloqueo de la cola: si la solicitud de la cima no tiene pista compatible, retrasa a las demás	Configurar pistas mixtas por defecto y, si hace falta, revisar las siguientes solicitudes de la cima
Integración tardía entre módulos	Interfaces de las clases definidas desde el inicio (esqueleto publicado en el repositorio) e integraciones parciales cada semana
Conflictos en Git por trabajo simultáneo	Una rama por integrante y revisión de cambios antes de unir a la rama principal
Valores simulados poco realistas	Documentar los parámetros en config/parametros.properties y ajustarlos tras las primeras pruebas

Declaración de uso de herramientas de inteligencia artificial

Herramienta	Uso realizado	Resultado incorporado	Forma de verificación
Claude (Anthropic)	Apoyo para estructurar el informe, proponer módulos y alternativas, y buscar fuentes académicas	Borrador de secciones, adaptado y ajustado por el equipo	Lectura de las fuentes citadas, revisión y discusión del contenido entre los integrantes
Claude (Anthropic)	Revisión técnica del informe: coherencia entre módulos y UML, comparador del heap, complejidades y formato APA	Diagrama de clases en PlantUML corregido, ajustes en las secciones 1 a 6 y esqueleto de clases del repositorio	Revisión del diagrama y de cada cambio por el equipo, compilación del esqueleto y comprobación de los DOI de las referencias

Referencias

Aguas Bucheli, L. F. (2024). *Data structures for everyone: Java exercises and theory*. Our Knowledge Publishing.

Beasley, J. E., Krishnamoorthy, M., Sharaiha, Y. M., & Abramson, D. (2000). Scheduling aircraft landings—The static case. *Transportation Science*, *34*(2), 180–197. https://doi.org/10.1287/trsc.34.2.180.12302

Bennell, J. A., Mesgarpour, M., & Potts, C. N. (2011). Airport runway scheduling. *4OR*, *9*(2), 115–138. https://doi.org/10.1007/s10288-011-0172-x

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms* (4.ª ed.). MIT Press.

Mesgarpour, M. (2012). *Airport runway optimization* [Tesis doctoral, University of Southampton]. https://eprints.soton.ac.uk/366012/

Odinsa. (s.f.). *Aeropuerto Internacional Mariscal Sucre*. Recuperado el 27 de octubre de 2026, de https://www.odinsa.com/en/airports/aeropuerto-internacional-mariscal-sucre/

Oracle. (s.f.). *Class PriorityQueue<E>* (Java SE 21 & JDK 21) [Documentación de API]. https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/PriorityQueue.html

Shirini, K., Aghdasi, H. S., & Saeedvand, S. (2024). A comprehensive survey on multiple-runway aircraft landing optimization problem. *International Journal of Aeronautical and Space Sciences, 25*(4), 1574–1602. https://doi.org/10.1007/s42405-024-00747-z

Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating system concepts* (10.ª ed.). Wiley.

Anexo A. Código PlantUML del diagrama de clases

Código fuente completo del diagrama de la Figura 1 (archivo `docs/uml/diagrama-clases.puml`). Se omiten aquí las líneas de estilo visual (`skinparam`).

```
@startuml diagrama-clases
' Simulador de gestión de pistas de un aeropuerto - Programación III (UDLA)
' Diagrama de clases preliminar - Progreso 1
hide circle
hide empty members
hide enum methods
title Simulador de gestión de pistas de un aeropuerto: diagrama de clases (Progreso 1)

package "modelo" {
  class Aeronave {
    - idVuelo : String
    - aerolinea : String
    - tipo : String
    - combustibleRestante : int
    - horaReporteCombustible : int
    + actualizarCombustible(minutos : int, hora : int) : void
    + horaAgotamiento() : int
    + estaBajoUmbral(umbral : int, horaActual : int) : boolean
  }

  abstract class Solicitud <<abstracta>> {
    - secuencia : long
    - aeronave : Aeronave
    - prioridad : Prioridad
    - prioridadOriginal : Prioridad
    - horaSolicitud : int
    + {abstract} getTipo() : TipoOperacion
    + {abstract} separacionRequerida(anterior : TipoOperacion) : int
    + calcularEspera(horaActual : int) : int
    + subirPrioridad(nueva : Prioridad) : void
    + compareTo(otra : Solicitud) : int
  }

  class SolicitudAterrizaje {
    + getTipo() : TipoOperacion
    + separacionRequerida(anterior : TipoOperacion) : int
  }

  class SolicitudDespegue {
    + getTipo() : TipoOperacion
    + separacionRequerida(anterior : TipoOperacion) : int
  }

  class Pista {
    - id : int
    - estado : EstadoPista
    - operacionesPermitidas : Set<TipoOperacion>
    - ultimaOperacion : TipoOperacion
    - horaUltimoInicio : int
    - horaLiberacion : int
    + estaDisponible(hora : int, s : Solicitud) : boolean
    + horaDisponible(s : Solicitud) : int
    + ocupar(a : Asignacion) : void
    + liberar() : void
    + cerrar() : void
    + reabrir() : void
  }
```

```
  class Asignacion {
    - solicitud : Solicitud
    - pista : Pista
    - horaInicio : int
    - horaFin : int
    + duracion() : int
    + tiempoEsperado() : int
  }

  class Alerta {
    - tipo : TipoAlerta
    - aeronave : Aeronave
    - hora : int
    - descripcion : String
  }

  enum Prioridad <<enumeración>> {
    EMERGENCIA
    COMBUSTIBLE_BAJO
    REGULAR
  }
  enum TipoOperacion <<enumeración>> {
    ATERRIZAJE
    DESPEGUE
  }
  enum EstadoPista <<enumeración>> {
    LIBRE
    OCUPADA
    CERRADA
  }
  enum TipoAlerta <<enumeración>> {
    EMERGENCIA
    COMBUSTIBLE_BAJO
    ESPERA_EXCEDIDA
  }
}

package "estructuras" {
  class ColaPrioridad {
    - heap : Solicitud[]
    - tamano : int
    + encolar(s : Solicitud) : void
    + extraer() : Solicitud
    + verSiguiente() : Solicitud
    + reordenar(s : Solicitud) : void
    + eliminar(s : Solicitud) : boolean
    + tamano() : int
    + estaVacia() : boolean
    - subir(i : int) : void
    - bajar(i : int) : void
  }

  class "PilaHistorial<T>" as PilaHistorial {
    - tope : Nodo<T>
    - tamano : int
    + apilar(dato : T) : void
    + desapilar() : T
    + verTope() : T
    + ultimos(k : int) : List<T>
    + estaVacia() : boolean
  }
}
```

```
package "servicio" {
  class Aeropuerto <<controlador>> {
    - horaActual : int
    + recibirSolicitud(s : Solicitud) : void
    + asignarSiguiente() : Asignacion
    + avanzarTiempo(minutos : int) : void
    + aplicarEnvejecimiento() : void
  }

  class RegistroSolicitudes {
    - aeronaves : Map<String, Aeronave>
    - activas : Map<String, Solicitud>
    + registrar(s : Solicitud) : boolean
    + cancelar(idVuelo : String) : Solicitud
    + buscarPorVuelo(idVuelo : String) : Solicitud
    + filtrar(criterio : Predicate<Solicitud>) : List<Solicitud>
    + clasificar(s : Solicitud, umbral : int) : Prioridad
  }

  class ControlCombustible {
    - umbralMinutos : int
    + declararEmergencia(s : Solicitud, motivo : String) : Alerta
    + actualizarCombustible(s : Solicitud, minutos : int, hora : int) : void
    + detectarBajoUmbral(pendientes : List<Solicitud>, hora : int) : List<Solicitud>
    + listarCriticos() : List<Solicitud>
    + generarAlerta(tipo : TipoAlerta, a : Aeronave, hora : int) : Alerta
  }

  class HistorialOperaciones {
    + registrar(a : Asignacion) : void
    + consultarUltimas(k : int) : List<Asignacion>
    + reportePorPista() : Map<Integer, Integer>
  }

  class Estadisticas {
    - totalOperaciones : int
    - sumaEspera : long
    - maxEspera : int
    - conteoPorPrioridad : Map<Prioridad, Integer>
    + registrar(a : Asignacion) : void
    + promedioEspera() : double
    + reportePorPrioridad() : Map<Prioridad, Double>
    + ordenarPorEspera(lista : List<Asignacion>) : List<Asignacion>
  }
}

' ---------- Herencia ----------
Solicitud <|-- SolicitudAterrizaje
Solicitud <|-- SolicitudDespegue

' ---------- Composición (el aeropuerto es dueño del ciclo de vida) ----------
Aeropuerto "1" *-- "1..*" Pista : pistas
Aeropuerto "1" *-- "1" ColaPrioridad : cola
Aeropuerto "1" *-- "1" RegistroSolicitudes : registro
Aeropuerto "1" *-- "1" ControlCombustible : control
Aeropuerto "1" *-- "1" HistorialOperaciones : historial
Aeropuerto "1" *-- "1" Estadisticas : estadisticas
HistorialOperaciones "1" *-- "1" PilaHistorial : pila
ControlCombustible "1" *-- "0..*" Alerta : alertas

' ---------- Agregación (las partes existen por sí solas) ----------
ColaPrioridad "1" o-- "0..*" Solicitud : pendientes
```

```
RegistroSolicitudes "1" o-- "0..*" Aeronave
PilaHistorial "1" o-- "0..*" Asignacion : completadas

' ---------- Asociaciones ----------
Aeronave "1" <-- "0..*" Solicitud : realizada por
Asignacion "0..1" --> "1" Solicitud
Asignacion "0..*" --> "1" Pista
Alerta "0..*" --> "1" Aeronave

' ---------- Dependencias con enumeraciones ----------
Solicitud ..> Prioridad
Solicitud ..> TipoOperacion
Pista ..> EstadoPista
Alerta ..> TipoAlerta

note right of Solicitud
  <b>compareTo(): comparador compuesto</b>
  1. Prioridad (EMERGENCIA > COMBUSTIBLE_BAJO > REGULAR)
  2. En COMBUSTIBLE_BAJO: menor horaAgotamiento()
  3. Menor horaSolicitud (= mayor espera)
  4. Menor secuencia (orden de llegada)
  Todas las claves son fijas mientras la solicitud
  está en el heap; si cambian, se llama reordenar().
end note
@enduml
```